Physics, Control, and Command Architecture

AI-Optimized Nanosatellite Constellation Manager – Phase 6

Technical Companion to the Python Simulation

October 16, 2026

Contents

1 Purpose and scope **2**

2 End-to-end data flow **2**

3 Orbital state from a TLE **2**

3.1 SGP4 propagation 2

3.2 From TEME to Earth-fixed coordinates 3

4 Mean orbit altitude used by the controller **3**

5 Station-keeping control loop **3**

6 Why a small burn raises an orbit **4**

7 Fuel use: the rocket equation **4**

8 Illustrative 24-hour controller output **4**

9 Phase 3: conjunction screening and virtual avoidance **5**

10 Phase 4: command-center dashboard **5**

10.1 Fleet scenarios and scale 6

11 Advisory AI/ML collision-risk scoring **7**

12 Telemetry dictionary **8**

13 Post-burn re-screen **8**

14 Model limits **8**

1 Purpose and scope

This document explains the physics represented by the project and how its code maps to an engineering workflow. The current implementation has four deliberately separate layers:

1. **Authoritative trajectory layer.** A Two-Line Element set (TLE) is propagated using SGP4. Its output is an inertial position and velocity in the True Equator Mean Equinox (TEME) frame, transformed to the Earth-fixed ITRS/ECEF frame for ground tracks.
2. **Control and resource layer.** A transparent, simplified mean-altitude model applies virtual orbital decay, commands an altitude-restoration burn, and tracks propellant through the rocket equation.
3. **Conjunction and command layer.** Every satellite pair is screened between samples with a cubic Hermite fit of the relative motion, and candidates are refined to the exact SGP4 time of closest approach. A deterministic, fuel-accounted virtual avoidance command is recorded whenever a selected keep-out zone is crossed.
4. **Advisory AI-risk layer.** A small, reproducible logistic model prioritizes deterministic candidates using relative motion. It cannot create, suppress, approve, or reject a safety action.

The control layer intentionally does *not* edit a TLE or claim to re-propagate an SGP4 orbit after a burn. This keeps the dashboard honest: the map is TLE/SGP4-derived, while the resource telemetry is a station-keeping demonstration model. A maneuver-aware numerical propagator is the appropriate later upgrade.

2 End-to-end data flow

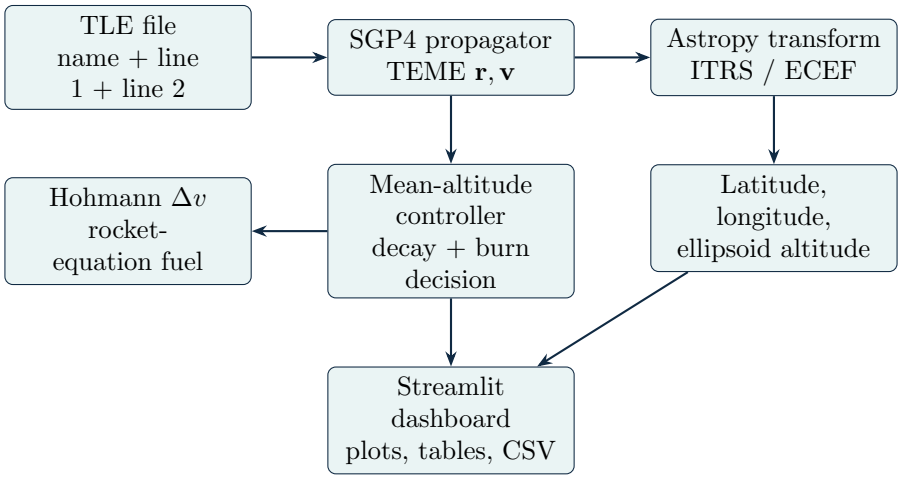


Figure 1: The trajectory and control models share time and satellite identity, but are intentionally distinguishable.

3 Orbital state from a TLE

3.1 SGP4 propagation

A TLE encodes mean orbital elements at a reference epoch. SGP4 uses those elements, Earth-gravity constants, and standard perturbation terms to calculate an Earth-orbiting object's state at a requested UTC epoch:

$$(\mathbf{r}_{\mathrm{TEME}}(t), \mathbf{v}_{\mathrm{TEME}}(t)) = \mathrm{SGP4}(\mathrm{TLE}, t). \tag{1}$$

The output units used by the Python `sgp4` package are kilometres and kilometres per second. SGP4 is particularly suitable for TLE-defined LEO tracking because the TLE itself is fitted to the SGP4 model.

3.2 From TEME to Earth-fixed coordinates

TEME is quasi-inertial, whereas a ground map must rotate with Earth. The program uses Astropy's time-aware coordinate transformation to form an ITRS/ECEF vector:

$$\mathbf{r}_{\mathrm{ECEF}}(t) = \mathbf{R}_{\mathrm{TEME}\to\mathrm{ITRS}}(t)\,\mathbf{r}_{\mathrm{TEME}}(t). \tag{2}$$

The transformation depends on Earth orientation and time, not simply a fixed z-axis rotation. The ECEF vector is converted to geodetic latitude ϕ, longitude λ, and ellipsoidal height h using WGS-84.

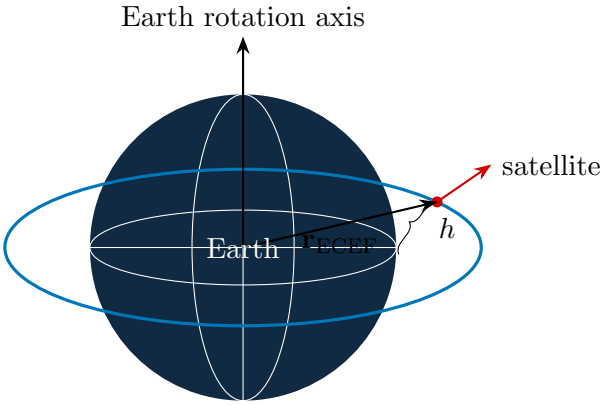


Figure 2: Earth-fixed geometry used by the ground-track view. The actual software computes the frame conversion rather than using this drawing's simplified geometry.

4 Mean orbit altitude used by the controller

Instantaneous geodetic altitude rises and falls around an eccentric orbit; it is therefore a poor direct trigger for station keeping. Instead, the controller derives an approximate mean orbit altitude from TLE mean motion n:

$$n_{\mathrm{rad/s}} = \frac{n_{\mathrm{rad/min}}}{60}, \tag{3}$$

$$a = \left(\frac{\mu_\oplus}{n_{\mathrm{rad/s}}^2}\right)^{1/3}, \tag{4}$$

$$h_{\mathrm{mean}} = a - R_\oplus, \tag{5}$$

where $\mu_\oplus = 398\,600.442\,\mathrm{km^3\,s^{-2}}$ and $R_\oplus = 6378.137\,\mathrm{km}$. This is a Keplerian mean-orbit estimate used only for the control state.

5 Station-keeping control loop

At each simulation step Δt, the virtual mean altitude loses a prescribed amount:

$$h_{k+1}^{-} = h_k - \dot{h}_{\mathrm{decay}}\frac{\Delta t}{86\,400.000\,\mathrm{s}}. \tag{6}$$

When h_{k+1}^{-} is below the permitted lower bound $h_{\mathrm{nominal}} - \epsilon$, a maneuver restores it to h_{nominal}. The UI exposes $\dot{h}_{\mathrm{decay}}$ and ϵ so that a short simulation can demonstrate a slow real-world process.

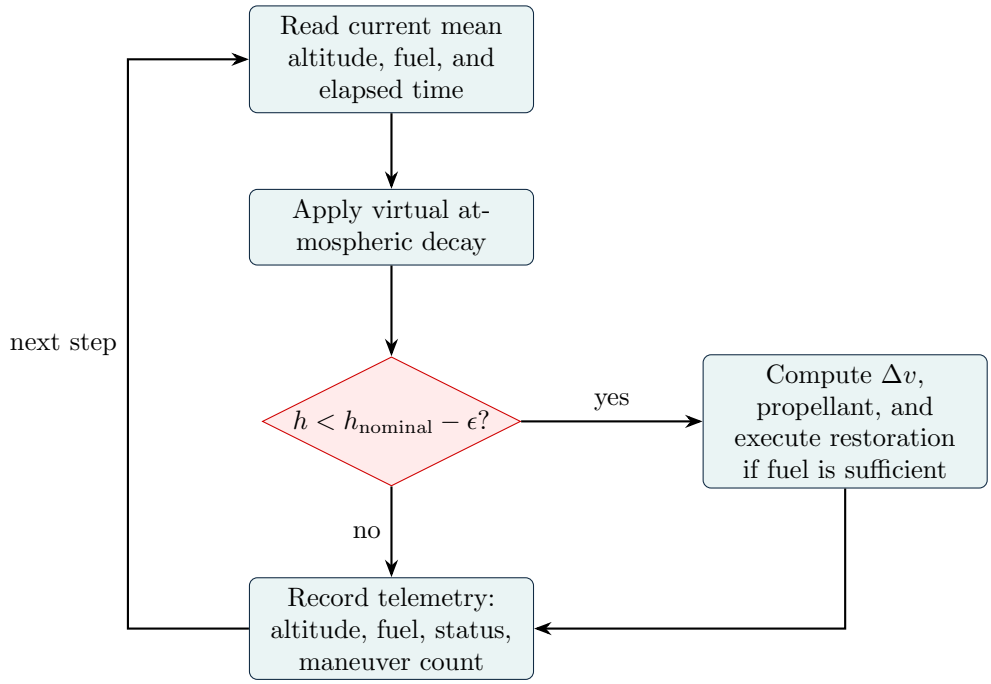


Figure 3: Closed-loop deterministic controller. Fuel exhaustion records `OUT_OF_FUEL` rather than pretending a maneuver occurred.

6 Why a small burn raises an orbit

For a prograde burn, speed rises at the current position. That point becomes the perigee of a new elliptical transfer orbit, whose apogee is higher. A second prograde burn at apogee circularizes the orbit. For a circular-to-circular Hohmann transfer from r_1 to r_2, the ideal total delta-v is:

$$\Delta v_1 = \sqrt{\frac{\mu}{r_1}}\left(\sqrt{\frac{2r_2}{r_1+r_2}} - 1\right), \tag{7}$$

$$\Delta v_2 = \sqrt{\frac{\mu}{r_2}}\left(1 - \sqrt{\frac{2r_1}{r_1+r_2}}\right), \tag{8}$$

$$\Delta v_{\text{total}} = |\Delta v_1| + |\Delta v_2|\,. \tag{9}$$

7 Fuel use: the rocket equation

The Tsiolkovsky rocket equation relates delta-v to a mass ratio:

$$\Delta v = I_{sp} g_0 \ln\left(\frac{m_0}{m_f}\right), \qquad m_f = \frac{m_0}{\exp\left(\Delta v/(I_{sp} g_0)\right)}. \tag{10}$$

The implementation uses $I_{sp} = 220.000\,\text{s}$, $g_0 = 9.807\,\text{m}\,\text{s}^{-2}$, dry mass $10.000\,\text{kg}$, and an initial propellant mass selected in the dashboard. Propellant consumed is $m_0 - m_f$. A maneuver is rejected if the computed consumption exceeds fuel remaining.

8 Illustrative 24-hour controller output

The following plots are a deterministic illustration using a decay rate of $1.000\,\text{km/day}$, a $0.200\,\text{km}$ tolerance, and hourly samples. Values are representative of the implemented equations; a run's exact values depend on each satellite's mean altitude.

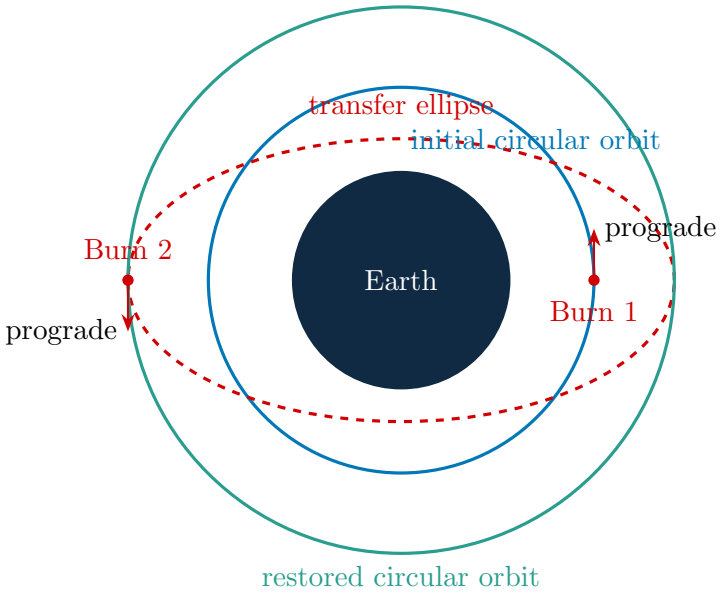


Figure 4: Idealized two-burn orbit raise. The project uses this as an energy-cost estimate, not as a high-fidelity maneuver propagator.

9 Phase 3: conjunction screening and virtual avoidance

For every unordered pair the software forms the relative position $\Delta\mathbf{r} = \mathbf{r}_i - \mathbf{r}_j$ and velocity $\Delta\mathbf{v}$ at each sample (TEME; the separation is the same in any frame). Sampling the separation alone misses most encounters: two LEO satellites close at 10–15 km/s, i.e. thousands of kilometres per 300 s step. Inside each interval $[t_k, t_k + h]$ the relative position is therefore represented by the cubic Hermite polynomial through the end values and end velocities,

$$\Delta\mathbf{r}(s) = h_{00}(s)\Delta\mathbf{r}_k + h\,h_{10}(s)\Delta\mathbf{v}_k + h_{01}(s)\Delta\mathbf{r}_{k+1} + h\,h_{11}(s)\Delta\mathbf{v}_{k+1}, \qquad s \in [0,1], \tag{11}$$

and its minimum norm d_k is found on a grid in s and polished by golden-section search. Every interval with $d_k < d_{\text{keepout}} + 25\,km$ is refined by golden-section search on the exact SGP4 separation, which gives the time of closest approach (TCA) and the miss distance d_{closest}. Adjacent intervals belonging to the same passage are merged into one event. On the archived ten-satellite fleet the Hermite estimate is within 0.4 km of the exact miss distance at a 300 s step, and the refined values agree with a 0.5 s brute-force search to a few metres.

The deterministic policy assigns one member of a flagged pair a retrograde phase-separation command. Its scale is a bounded approximation based on additional clearance accumulated over planning time T:

$$\Delta v_{\text{avoid}} = \text{clip}\left(\frac{1000\,(1.25 d_{\text{keepout}} - d_{\text{closest}})}{T}, \Delta v_{\min}, \Delta v_{\max}\right). \tag{12}$$

The command consumes fuel using the same rocket equation as station keeping. It is labelled `MANEUVER_PLANNED`, rather than a verified collision resolution, until the post-burn orbit is numerically propagated.

10 Phase 4: command-center dashboard

The Streamlit command center turns the simulation table into an operator-oriented interface. A mission-time selector creates one coherent UTC fleet snapshot, so every map marker, orbital marker, and status row refers to the same instant.

1. **Overview:** fleet altitude, resource health, and active alerts.

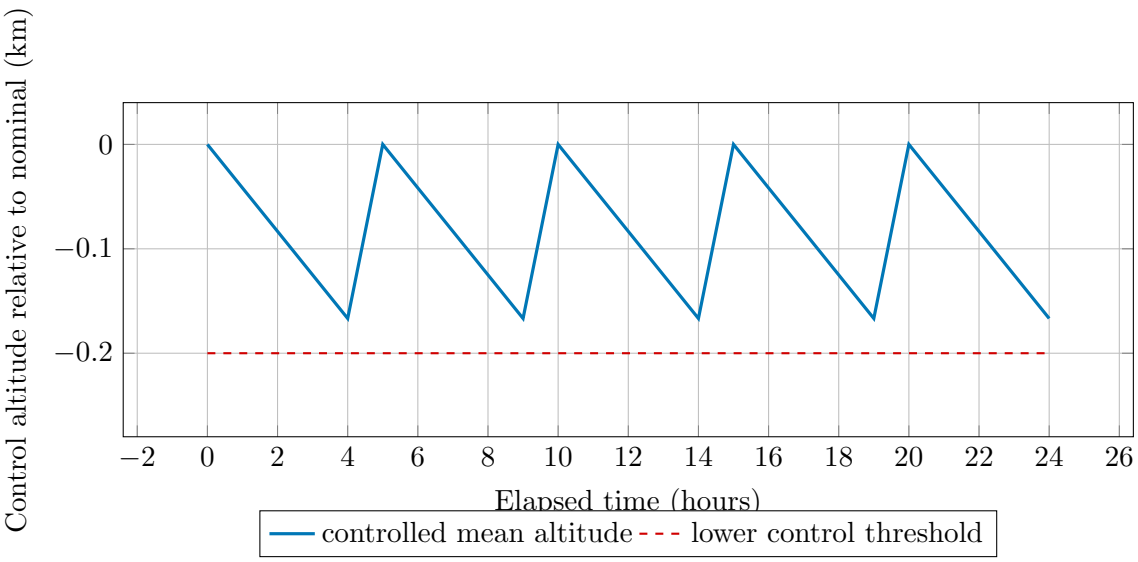


Figure 5: Sawtooth behavior of the mean-altitude controller. The next hourly sample would cross the threshold and trigger another restoration.

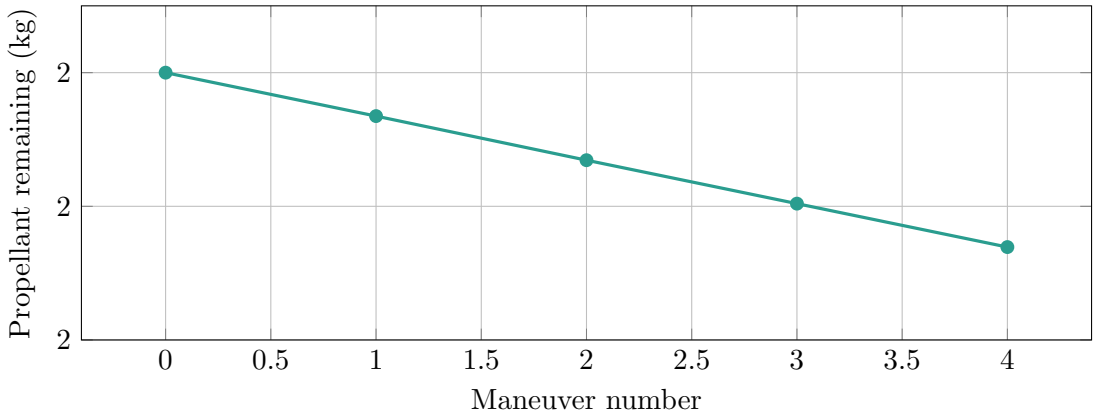


Figure 6: Illustrative propellant depletion for a low-thrust station-keeping sequence. Fuel declines monotonically and never becomes negative.

2. **Ground tracks:** geodetic latitude/longitude paths with selected-time markers.
3. **3D orbital web:** TEME trajectories around a reference Earth sphere.
4. **Telemetry and alerts:** resources, maneuvers, conjunction fields, and complete CSV export.

10.1 Fleet scenarios and scale

The dashboard offers a five-object sample and a ten-object archived operational-satellite scenario. The latter contains ISS, HST, Terra, Aqua, Suomi NPP, Landsat 8, Landsat 9, Sentinel-1A, Sentinel-2A, and NOAA-20. Its TLEs are a static CelesTrak active-catalogue snapshot with epoch day 2026–220, retained in the repository for reproducibility rather than fetched at run time. Accordingly, the data are not live tracking information and must never be treated as operational ephemerides.

For N spacecraft, the same-timestamp conjunction screen evaluates $N(N-1)/2$ unordered pairs. The five-object sample has 10 comparisons per timestamp; the ten-object fleet has 45. The dashboard displays this count with the selected scenario so that a future larger synthetic fleet can be assessed with an appropriate timestep and display filtering strategy.

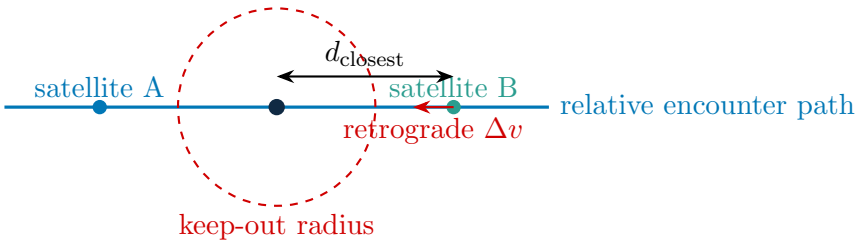


Figure 7: Conceptual conjunction screen and virtual phase-separation command.

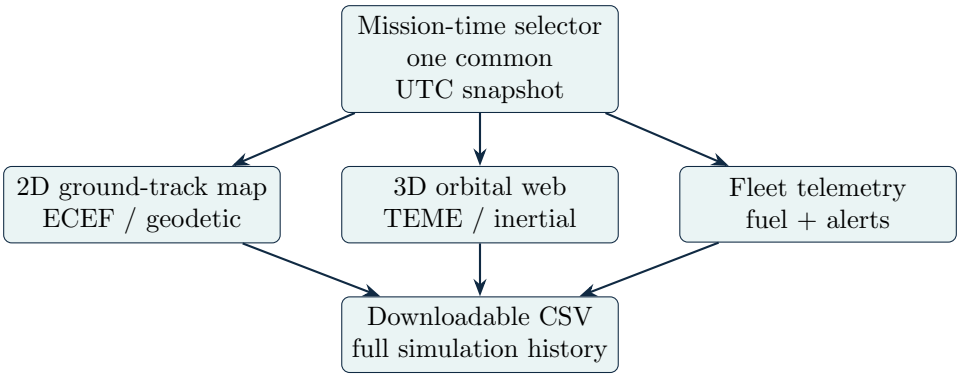


Figure 8: Phase 4 command-center interaction model.

11 Advisory AI/ML collision-risk scoring

The risk score is deliberately lightweight and fully reproducible: it is a fixed logistic-regression-style model with no live feeds, external services, stochastic training, or hidden state. Its coefficients were calibrated against synthetic relative-motion examples solely to provide a smooth, interpretable priority signal in this simulation. For a relative TEME position $\Delta\mathbf{r}$, relative TEME velocity $\Delta\mathbf{v}$, and linearized time to closest approach,

$$t_{\text{TCA}} = \max\left(0, -\frac{\Delta\mathbf{r}\cdot\Delta\mathbf{v}}{\|\Delta\mathbf{v}\|^2}\right), \tag{13}$$

$$d_{\text{pred}} = \|\Delta\mathbf{r} + t_{\text{TCA}}\Delta\mathbf{v}\|. \tag{14}$$

The model receives $\Delta\mathbf{r}$, $\Delta\mathbf{v}$, t_{TCA}, d_{pred}, and the configured keep-out distance d_k. It maps normalized proximity, current distance, relative speed, and temporal urgency through a sigmoid $p = (1 + e^{-z})^{-1}$. The resulting p is labelled `LOW`, `ELEVATED`, `HIGH`, or `CRITICAL`. It is a calibrated-for-simulation priority score, not a physical probability of collision.

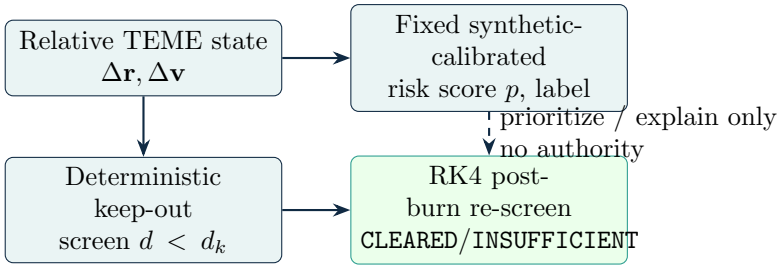


Figure 9: Safety architecture. The dashed advisory path cannot override deterministic screening or numerical verification.

The CSV, per-satellite telemetry, and conjunction log preserve `relative_position_km`, `relative_velocity_k`
`time_to_closest_approach_s`, `predicted_closest_separation_km`, `keep_out_distance_km`, `ai_risk_probabili`

and `ai_risk_label`. They also include `ai_primary_driver` and `ai_model_version`, making the priority explanation and fixed model revision auditable. The synthetic dashboard lab reports the score before its unsafe intercept and after its maneuvered pass alongside their separations.

12 Telemetry dictionary

Field	Unit	Meaning
`altitude_km`	km	Instantaneous ellipsoidal altitude from ECEF/ITRS
`mean_orbit_altitude_km`	km	Mean-altitude estimate derived from TLE mean motion
`controlled_mean_altitude_km`	km	Virtual controller state after decay/burn action
`maneuver_delta_v_m_s`	m/s	Delta-v spent at the current step
`cumulative_delta_v_m_s`	m/s	Total delta-v spent by that satellite
`fuel_remaining_kg`	kg	Remaining virtual propellant
`station_keeping_status`	–	`NOMINAL`, `MANEUVER`, or `OUT_OF_FUEL`
`conjunction_alert`	–	Whether this satellite belongs to a flagged close-approach pair
`closest_approach_km`	km	Miss distance at the refined time of closest approach
`avoidance_status`	–	`CLEAR`, `MANEUVER_PLANNED`, or `OUT_OF_FUEL`
`ai_risk_probability`	0–1	Advisory synthetic-calibrated encounter-priority score
`ai_risk_label`	–	`LOW`, `ELEVATED`, `HIGH`, or `CRITICAL`
`ai_primary_driver`	–	Largest positive fixed-model contribution to the priority score
`ai_model_version`	–	Immutable scoring-model revision identifier

13 Post-burn re-screen

A planned burn is checked by propagating the manoeuvring satellite's TEME state from the burn time to the TCA twice with a fourth-order Runge–Kutta two-body integrator, once with and once without the velocity change. Only the *difference* of the two arcs, $\delta\mathbf{r}$, is used:

$$d_{\text{post}} = \left\| \mathbf{r}^{\text{SGP4}}_{\text{man}}(t_{\text{TCA}}) + \delta\mathbf{r} - \mathbf{r}^{\text{SGP4}}_{\text{prot}}(t_{\text{TCA}}) \right\| . \tag{15}$$

Comparing a two-body arc directly with the other satellite's SGP4 position would mix in the difference between the two force models, which on this fleet is 28–430 km after six hours, far larger than the effect of a sub-m/s burn (about 0.7 km per 0.01 m/s over six hours). With the differential form a zero burn reproduces the SGP4 miss distance exactly.

14 Model limits

The present model is appropriate for demonstrating systems engineering connections: TLE ingestion, frame conversion, feedback control, delta-v budgeting, conjunction screening and a dashboard. It is not a mission-design-grade orbit determination system. It does not include drag density models, covariance propagation, collision probability, thrust direction errors or eclipse constraints, and the burn effect uses two-body dynamics.

The AI layer adds further limits: it has no tracking covariance, object size, attitude, maneuver uncertainty, high-fidelity force model, sensor data, or operational calibration data. Linearized TCA is only a feature, not an orbit propagator. Therefore this software is a simulation and decision-support demonstrator, *not* an operational collision-avoidance system; humans and validated deterministic flight-dynamics processes must retain authority.